SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

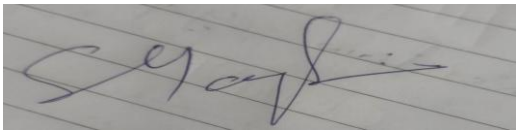
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

*I **am Manoj Kumar S /192311095** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary (5 Marks)**	**Level 3 – Proficient (4 Marks)**	**Level 2 – Developing (2–3 Marks)**	**Level 1 – Beginning (0–1 Mark)**
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1

A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer

Concept

Testing as an engineering activity means that testing is not an informal "try and see" task. It is planned, designed, executed, measured and continuously improved using defined methods, tools and standards, exactly as other engineering work is. Random testing depends on the tester's memory and intuition, so it is not repeatable, not measurable and leaves no evidence of what was covered.

Explanation

- **Test planning:** Define scope, objectives, risks, test levels, resources, schedule and entry/exit criteria.
- **Test analysis and design:** Study the requirements and derive test cases with techniques such as equivalence partitioning, boundary value analysis, decision tables and state transition testing.
- **Test implementation and execution:** Prepare data and environment, run the documented cases and record actual results.
- **Defect management:** Log every failure in a tracker (for example Jira) with a standard report and follow it until closure.
- **Evaluation and improvement:** Use metrics such as requirement coverage, defect density and defect detection rate to decide when to stop and what to improve.
- **Tools and standards:** Test management and automation tools (TestRail, Selenium, JUnit) and IEEE 829/ISO 29119 templates.

How structured test design improves quality and defect detection

- **Coverage:** Each requirement is linked to test cases (traceability), so no feature is skipped.
- **Effectiveness:** Techniques choose the values most likely to expose defects, so a small set finds more defects than many random tests.
- **Repeatability:** Another tester can run the same case and get comparable results.
- **Measurement:** Management can see what is tested and how good the product is.

Application to the Scenario

For a startup's signup form with an age field accepting 18 to 60, a random tester tries 25 and 40 and sees no problem. Structured design produces the partitions (below 18, 18 to 60, above 60) and the boundary values 17, 18, 19, 59, 60 and 61. If the code uses "age > 18", the value 18 is wrongly rejected, and the defect is detected with only six designed cases.

Conclusion

By redesigning testing as a planned, documented and measured engineering process, the startup obtains consistent results, higher defect detection and a product of predictable quality.

Question 2

A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Answer

Concept

A testing process is a defined and documented sequence of activities: test planning, test design, test environment set-up, execution, defect reporting, evaluation of exit criteria and test closure. It is supported by roles, templates, tools and metrics, and is followed by every team in the same way. In real-time systems such as banking, where wrong results mean financial loss and loss of trust, a defined process is essential.

Explanation

- **Planning:** Identifies the risks (money transfer, interest, limits) and the test levels needed.
- **Design:** Test cases are derived from the business rules and reviewed before use.
- **Execution and reporting:** Results are recorded and every failure is logged, analysed and retested.
- **Exit criteria:** The release is allowed only when agreed conditions are met, for example no open critical defects and 100% of high-priority cases passed.
- **Closure:** Lessons learned and defect data are stored for the next release.

Benefits with examples

Benefit	Example in a banking system
Fewer failures in production	A designed case for "transfer greater than balance" confirms that the transaction is rejected
Early defect detection	Review of the interest rule finds an ambiguous requirement before coding
Repeatability	The same regression suite is run on every release, so a fixed defect does not return
Data integrity	A test that forces a failure after debit and before credit proves the rollback works
Better control and trust	Exit criteria and metrics give managers reliable release decisions

Application to the Scenario

Before the process, testers checked only simple transfers and missed concurrency problems. With the defined process, a concurrency test (two withdrawals at the same instant on the same account) is part of the plan, and an overdraft defect is found before release.

Conclusion

A well-defined process makes testing systematic, repeatable and measurable, and so reduces failures, cost and risk in banking systems.

Question 3

During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer

Concept

In a structured process, testing is integrated into every SDLC phase instead of being a single activity at the end. Each development phase has a matching verification or validation activity (the V-model), so that defects are found in the phase in which they are introduced.

Explanation

SDLC phase	Testing activity	E-commerce example
Requirements	Requirement review, testability check, test planning	Clarify discount, tax and return rules
Design	Design review, test case and data design	Check cart, payment and inventory interfaces
Coding	Unit testing, code review, static analysis	Test the discount calculation function
Integration	Integration testing of modules and interfaces	Cart with payment gateway; order with inventory
System	Functional, performance, security and usability testing	Complete purchase flow with 1,000 users
Acceptance / Maintenance	User acceptance testing, regression testing	Business users confirm; retest after each update

How process discipline ensures better quality

- Defects are detected where they originate, so fewer pass into integration and system testing.
- Entry and exit criteria prevent an unstable build from moving to the next phase.
- Standard templates and reviews give the same level of testing in every phase and every module.
- Traceability shows which requirement is covered and which is not.
- Metrics provide the data for decisions and for improvement.

Application to the Scenario

If the payment interface is reviewed at the design stage and tested in integration, a mismatch in the amount format between the cart and the gateway is found early, and does not appear as a failed order in system testing or in production.

Conclusion

Disciplined, phase-wise testing reduces integration and system defects, lowers the cost of correction and results in a more reliable platform.

Question 4

A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer

Concept

- **Error (mistake):** A human action that produces an incorrect result, for example a developer misreading a requirement.
- **Defect (fault, bug):** The flaw in the code or document caused by the error; it is the cause present in the software.
- **Failure:** The visible deviation of the system from its expected behaviour when the defect is executed.

Explanation

The relationship is a chain: Error, then Defect, then Failure. A person makes an error; this error places a defect in the work product; and when the defective part is executed under suitable conditions, the system fails. A defect does not always cause a failure: if the faulty code is never executed, or the input never triggers it, the defect remains hidden. Also, a failure may come from the environment or from wrong test data, so it should be analysed before it is called a defect.

Application to the Scenario

Item	Classification in the login system
Mismatch seen by the tester	Failure (expected: login rejected; actual: login accepted)
Faulty code	Defect: password comparison converts both values to lower case
Root mistake	Error: developer assumed that passwords are not case-sensitive, although the requirement says they are

Example: the correct password is "Abc@123". The tester enters "abc@123" and the system accepts it. This observed mismatch is the failure. The team then searches the code, finds the lower-case conversion (defect) and traces it to the developer's misunderstanding (error).

Conclusion

The mismatch observed during validation is a failure. Using the three terms correctly makes defect reports clear, helps root-cause analysis and avoids wasted effort.

Question 5

A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer

Concept

The principle applied is Early Testing. It states that testing activities should begin as early as possible in the software life cycle, starting with the review of requirements and design, and not wait until the code is complete. The cost of fixing a defect increases sharply with each phase in which it remains undetected, because requirements, design, code and test documents all have to be changed.

Explanation

- Requirement reviews and walkthroughs remove ambiguity, omissions and contradictions.
- Testers prepare the test plan and test cases while requirements are written, which exposes unclear points.
- Design reviews and inspections find interface and logic problems before coding.
- Static analysis and unit tests give feedback within hours of coding.
- Continuous integration runs automated checks on every build.

How it improves effectiveness

- Defects are found when they are cheapest to correct.
- Rework, delay and total project cost are reduced.
- Test cases are ready by the time the software is built, so the testing phase is shorter.
- Communication between customer, developer and tester improves.

Application to the Scenario

A requirement says "the report must load quickly". A tester in the review asks for a measurable value and the requirement is changed to "within 3 seconds for 1,000 records". If this had been discovered in system testing, the design and code would need rework at a cost many times greater.

Conclusion

Applying early testing improves defect detection efficiency, reduces later-stage cost and increases the overall quality of the product.

Question 6

During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer

Concept

Requirement-phase testing is a form of static testing (verification). The tester examines the requirement documents, not the running software, to make sure the requirements are complete, clear, consistent and testable. Requirement defects are the most expensive kind, because they affect design, code and tests.

Explanation

- **Review for ambiguity:** Words such as "quickly", "appropriate" or "as required" are questioned and made measurable.
- **Check completeness:** Missing cases such as patient allergies, duplicate patient records, invalid age or emergency access are identified.
- **Check consistency and testability:** Conflicting rules are resolved, and each requirement must have a verifiable acceptance criterion.
- **Risk analysis:** High-risk functions (dosage calculation, prescription, patient data) are identified for deeper testing.
- **Preparation:** Draft the test plan, acceptance criteria and a requirement traceability matrix.
- **Communication:** Raise questions with analysts, doctors and developers in review meetings.

Importance of early involvement

- Errors found in requirements cost the least to correct.
- Clear requirements lead to correct design and code, and so fewer defects later.
- Test design starts early, shortening the later test phase.
- In healthcare, early detection protects patient safety and regulatory compliance.

Application to the Scenario

The requirement "the system shall calculate the correct medicine dose" is unclear. The tester asks for the formula, the units and the limits for children and adults. The requirement is rewritten with these details, so the developer implements the correct logic and the tester can design boundary tests for weight and age.

Conclusion

Early involvement of testers in requirement analysis prevents ambiguities from becoming defects, and improves the quality, safety and cost-effectiveness of the healthcare system.

Question 7

A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer

Concept

A defect originates when a human error is introduced into a work product in some phase of the life cycle. Defects can be injected in the requirement, design, coding, testing and maintenance phases, but a defect injected early spreads to all the later phases that depend on it.

Explanation

Phase	How the defect originates in the payroll system
Requirements	Ambiguous statement such as "deduct tax as applicable", with no slabs, rounding or exemption rules (error by the analyst)
Design	Developer designs the formula on an assumption; a salary field is given an integer data type
Coding	The assumption is coded, for example "income > 5,00,000" instead of ">=", causing a wrong deduction
Testing	Test cases are written from the same unclear requirement, so the wrong output passes unnoticed
Maintenance	A tax-rule change is applied to one module only

Thus, one error in the requirement becomes a defect in design and code and ends as a failure: an incorrect salary slip. Analysis of the defect repository (origin phase) and root-cause techniques such as 5 whys identify the phase where the defect was injected.

How requirement clarity prevents such issues

- Each rule is stated in a precise, measurable way, with formulas, limits, units and rounding.
- Worked examples are included, for example "gross Rs.50,000 with 10% tax gives net Rs.45,000".
- Requirements are reviewed jointly by analysts, developers, testers and payroll experts.
- A traceability matrix links each requirement to design, code and tests, and a change process controls modifications.

Conclusion

Clear, complete and reviewed requirements remove the assumptions that lead to defects, so the payroll system produces correct outputs and rework is reduced.

Question 8

A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Answer

Concept

Defects can be classified by origin, type, severity and priority. Boundary-related defects arise because the programmer handles the edge of a valid range incorrectly, for example by using < instead of <=, making an off-by-one error in a loop, or not handling the maximum length of a field.

Classification and justification

Basis	Classification	Justification
Type	Coding / logic defect (input handling and boundary defect)	The failure occurs only at the edges of the input range, which points to wrong conditions in the code
Origin	Coding phase (possibly design)	The requirement gave the limits; the implementation did not handle them correctly
Severity	Critical / major	The system crashes
Priority	High	Users can reach the failure with ordinary input

How boundary value analysis (BVA) could have prevented it

BVA is a black-box technique based on the observation that defects occur most often at the edges of input ranges. For each boundary, the tester selects the value just below, exactly on and just above it. When combined with equivalence partitioning, BVA covers each class with a few well-chosen values.

Application to the Scenario

For a quantity field accepting 1 to 100, BVA gives 0, 1, 2, 99, 100 and 101. The crash at 100 or 101 would have been detected during testing with these six cases, and the code would have been corrected before release. The team suspects improper coverage because only typical values such as 10 or 50 were tested.

Conclusion

The defect is a high-severity coding (boundary) defect. Using BVA in test design would have given complete edge coverage with very few test cases, preventing the crash.

Question 9

A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer

Concept

A defect repository is a central, structured database in which all defects are recorded with details such as ID, module, type, severity, priority, origin phase, root cause, status, fix and dates. It is the organisation's memory of its failures, and if analysed, it shows where and why defects occur.

Support for better test case design

- **Defect clustering:** Modules with a high defect count are identified, and more and deeper test cases are designed for them.
- **Regression cases:** Every fixed defect is converted into a regression test so that it cannot return unnoticed.
- **Defect patterns:** If many defects are boundary or validation errors, such cases are added to the design checklist of every new module.
- **Prioritisation:** Severity and frequency data decide which test cases run first.
- **Reviews:** Past defects are used to build review checklists for requirements, design and code.
- **Estimation:** Historic defect density helps estimate the effort needed for the next release.

Role in improving software quality

- Enables root-cause analysis and so process improvement.
- Prevents repeated defects across releases.
- Provides metrics such as defect density, defect removal efficiency and reopen rate.
- Supports release decisions and training of new testers.

Application to the Scenario

Suppose analysis shows that 40% of defects in the last three releases were input validation errors in the Registration module. The team adds validation and boundary cases for all form fields, introduces a validation checklist for code review, and defects of this type fall in the next release.

Conclusion

A defect repository that is actively analysed turns past failures into better test cases and better practices, and so steadily improves product quality.

Question 10

An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer

Concept

A functional defect is a defect in which the system does not perform a function in the manner specified by the requirements. It is found by black-box (functional) testing against the expected result.

Identification as a functional defect

The specified function of the withdrawal is: validate the card and PIN, check the balance and daily limit, debit the account, dispense the same amount of cash and print the receipt. The failure occurs because the transaction logic is incorrect: for example the account is debited but cash is not dispensed when the network times out, the daily limit is ignored, or the balance check uses "greater than" instead of "greater than or equal to". The behaviour deviates from the specified function, so it is a functional (logic) defect of critical severity, because it directly affects customers' money.

How it could have been prevented

- Requirement and design review to define all rules and failure handling (timeout, cash jam, power loss).

- Decision table testing for the combinations of balance, limit, PIN status and cash availability.
- **Boundary value tests:** balance equal to the amount, amount equal to the limit, limit plus Rs.100.
- State transition testing of the withdrawal flow, and negative tests such as wrong PIN three times.
- Unit tests and code review of the transaction function, and integration tests with the bank server for rollback.
- Regression testing after each change, and traceability to ensure every rule has a test.

Application to the Scenario

A test case "balance Rs.5,000, withdraw Rs.5,000, expected: success and balance Rs.0" would have exposed the wrong operator at once.

Conclusion

The issue is a critical functional defect which systematic, requirement-based test design and reviews would have detected before release.

Question 11

A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer

Concept

Defect clustering is a testing principle which states that a small number of modules usually contain most of the defects, in line with the Pareto principle (about 80% of defects come from about 20% of modules). The causes include high complexity, frequent changes, new technology and developer inexperience.

Identifying testing priorities

- Collect defect data per module from the defect repository and compute defect density.
- Rank modules by defect count, complexity, change frequency and business criticality.
- Allocate the highest priority, the most experienced testers and the deepest techniques (boundary, decision table, white-box coverage) to the top-ranked modules.
- Run the critical tests of the clustered modules first and repeat them after each fix.
- Use lighter, automated regression for stable modules with few defects.
- Review the ranking every release, because clusters can move.

How efficiency improves

- More defects are found for each hour of testing, because effort is spent where defects are likely.
- Critical defects are detected early, so developers have time to correct them.
- Cost and schedule delay are reduced, and release risk is lowered.
- Resources are not wasted on exhaustive testing of low-risk modules.

Application to the Scenario

In an online store, 70% of defects come from Payment and Cart. The plan gives 60% of the effort to these two modules with extra boundary, failure-recovery and concurrency tests, while Catalogue and Help receive automated regression only.

Conclusion

Using defect clustering, the team prioritises tests by risk and defect history, so that the same effort gives higher defect detection and better product quality.

Question 12

A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer

Concept

Defect classification groups defects by common attributes such as type, origin, severity, priority and detection phase. Standard schemes include the IEEE 1044 classification. Classification makes defects easier to analyse, assign and prevent.

Classification of the slow-performance defect

Basis	Classification	Reason
Type	Non-functional: performance (load/scalability) defect	The function works, but response time is not acceptable under load
Probable origin	Design or coding	Inefficient database queries, no caching, small connection pool, memory leak
Severity	Major to critical	Many users are affected at the peak
Priority	High	It affects business during busy periods
Detected by	Load / stress testing or monitoring	Needs many simulated users

How classification helps resolution

- **Assignment:** The defect is routed to the right specialists (database or performance engineers).
- **Correct testing:** It tells the team to use performance tools such as JMeter or LoadRunner, and not functional tests.
- **Prioritisation:** Severity and priority decide the order of fixing.
- **Root cause:** Origin data show where to look, for example queries or server configuration.
- **Trend analysis:** Many performance defects show that non-functional requirements and early load testing are missing.

Application to the Scenario

A load test with 2,000 users shows the response time rising from 2 to 15 seconds. It is classified as a performance defect; profiling shows a missing database index; adding the index and re-running the test confirms the response time is below 3 seconds.

Conclusion

Proper classification gives the defect clear ownership, priority and a suitable resolution method, and produces data to prevent similar defects.

Question 13

A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer

Concept

Regression testing is the repetition of tests after a change to confirm that the change has not damaged existing functionality and that previously fixed defects have not returned. Running every test after every release is costly, so the suite must be chosen intelligently. Historical defect data is the best guide for this choice.

Reuse of historical defect data

- **Defect-based test cases:** Each previously fixed defect is turned into a test case that verifies the fix.
- **Risk-based selection:** Modules with a high defect density or frequent changes are included in every regression run.
- **Prioritisation:** Tests connected to critical and major defects are run first.
- **Pattern-based additions:** If past defects show repeated boundary or integration errors, similar checks are added to the suite.
- **Suite maintenance:** Tests for stable areas with no defects over many releases are run less often, and obsolete tests are removed.
- **Traceability:** Defect-to-test links show which test to run when a given code area changes (impact analysis).

Improvement in coverage and efficiency

- **Coverage:** The suite contains real situations in which the product has failed before, so important risk areas are covered.
- **Efficiency:** Limited time is used on high-risk areas, not on re-running everything.
- **Automation:** Stable, repeated cases are automated for fast feedback in continuous integration.
- Reduced escape of old defects into production.

Application to the Scenario

If the Payment module caused 30 of the last 50 defects, the fixed-defect tests for Payment form the core of the regression pack and are run on every build, while the Help pages are tested once per release.

Conclusion

Historical defect data makes regression testing targeted and risk-based, giving better coverage in less time.

Question 14

During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer

Concept

Integration defects arise at the interfaces between modules: different data formats, wrong parameters, mismatched assumptions about responsibilities or timing. These are mostly caused by misunderstanding between team members, so communication and early tester involvement are key to reducing them.

Role of the tester in minimizing such defects

- Review requirements and interface specifications with developers to clarify data types, formats and error handling.
- Take part in design reviews, code reviews and sprint planning meetings.
- Prepare integration test cases and test data early, and share them with the developers.
- Report defects clearly (steps, expected and actual results, logs), and confirm fixes quickly.
- Maintain a shared defect tracker, test status and traceability matrix.
- Provide early feedback through smoke tests of every build.

How early involvement helps

- Misunderstandings are discovered in design reviews, and not in integration testing.
- Developers know in advance how the product will be tested, and testers understand how it is built.
- Shared understanding reduces rework and the number of defect reports and rejected defects.
- Trust and collaboration improve, and the schedule becomes more predictable.

Application to the Scenario

The Order module sends the date as DD/MM/YYYY, but the Billing module expects YYYY-MM-DD. A tester who attends the interface design review notes the mismatch, and the format is agreed before coding. Without this, the defect would appear later as failed invoices in integration testing.

Conclusion

Active, early participation of testers and good collaboration practices such as joint reviews and shared tools greatly reduce integration defects.

Question 15

A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Answer

Concept

Defect prevention means identifying the causes of defects and changing the process so that the same defects are not introduced again, rather than only detecting and fixing them. It is based on analysis of the defect repository, and is a core part of quality assurance.

Defect prevention techniques

- **Root-cause analysis:** Use the repository, 5-whys and fishbone diagrams to find why a defect type recurs.
- **Process improvement:** Change the activity that caused the defect, for example add a design review or an interface checklist.
- **Reviews and inspections:** Perform requirement, design and code reviews using checklists built from past defects.
- **Standards and training:** Apply coding standards and static analysis tools, and train developers on frequently repeated errors.
- **Regression and automation:** Add a test for every fixed defect, run in continuous integration.
- **Retrospectives:** Review defect trends after every release and track corrective actions to completion.

How prevention improves quality

- Removes the cause and not only the symptom, so defects do not return.
- Reduces rework, cost and time to release.
- Increases reliability and customer satisfaction.
- Creates a proactive quality culture and measurable improvement (falling defect density).

Application to the Scenario

If analysis shows that most recurring defects are null-pointer errors due to unvalidated input, the team adds an input validation standard, a static analysis rule, a review checklist item and regression tests. In the next release these defects no longer appear.

Conclusion

Defect prevention provides a long-term solution because it improves the process itself and so raises the overall quality of every future release.

Question 16

A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.

Answer

Concept

A test strategy is a high-level description of how testing will be carried out: the test levels, techniques, tools, environments and standards. For a mobile application, it must also address the special factors of devices, operating systems, screen sizes, network conditions, battery and memory, and interruptions.

Redesigned strategy based on testing principles

- **Early testing:** Review requirements and design; developers write unit tests for each component.
- **Defect clustering and risk-based testing:** Identify the crash-prone features from crash reports and test them first and deepest.
- **Exhaustive testing is impossible:** Select devices and OS versions from usage analytics and use a compatibility matrix.
- **Pesticide paradox:** Review and update the test suite regularly, adding a case for every crash fixed.

- **Standard process:** Common test plan, test case and defect templates, and exit criteria.
- **Mobile-specific tests:** Low memory, network loss, incoming calls, rotation, installation and upgrade, using real devices and a device cloud (Appium, Firebase Test Lab).
- **Non-functional tests:** Performance, memory-leak and battery tests; monitoring of crashes in production with crash analytics.

How structured approaches improve reliability

Every release receives the same level of testing, results are repeatable and comparable, and each crash is traced to a root cause and covered by a regression test, so it does not recur. Automated regression in continuous integration gives quick feedback, and the application becomes stable and predictable.

Application to the Scenario

A crash on low-memory phones during image upload is found by testing on a low-end device in the matrix, corrected by releasing bitmaps, and added to the regression suite.

Conclusion

Standardised, principle-based testing finds crashes before release and gives a reliable mobile application.

Question 17

A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

Answer

Concept

- **Testing:** The process of executing software (and reviewing work products) with the intention of finding defects and checking whether it meets the requirements.
- **Debugging:** The process of finding the cause of a known failure, correcting the defect and verifying the correction.

Differences

Aspect	Testing	Debugging
Purpose	Detect failures and show conformance to requirements	Locate and remove the cause of a failure
Performed by	Testers (and developers)	Developers
Input	Requirements, test cases	Defect report, failing test, logs
Planning	Planned, designed and scheduled	Depends on defects reported
Output	Test results and defect reports	Corrected code
Automation	Largely automated (regression)	Mainly manual, with debugger tools

Application to the Scenario

Online payment scenario: the tester enters an amount of Rs.999.50 and the order total shows Rs.999. Testing has detected the failure, and a defect report is raised with the steps, expected result and actual result. The developer reproduces it, uses the debugger and finds that the amount is stored in an integer variable (debugging). The variable is changed to decimal and unit tested. The tester performs confirmation testing of the fix and regression testing.

Roles in improving software quality

Testing measures quality and makes defects visible; debugging improves quality by removing the defects. They are different but complementary steps of one cycle: test, report, debug, fix, retest. If testing is confused with debugging, developers test only what they expect, defects are missed and testers wait while causes are located, so the release is delayed.

Conclusion

Clear separation of roles, with a defined hand-over through defect reports, makes defect resolution faster and quality higher.

Question 18

An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.

Answer

Concept

Cross-browser testing (a type of compatibility testing) verifies that a web application works correctly and looks consistent on different browsers, versions, operating systems and devices. Differences arise because browsers use different rendering engines (Blink, Gecko, WebKit), support different features of HTML, CSS and JavaScript, and have different default settings.

Cross-browser testing strategy

- **Define scope:** Use web analytics to choose the browsers and versions, devices and operating systems of real customers (Chrome, Edge, Firefox, Safari on Windows, macOS, Android, iOS).
- **Risk-based priority:** Test the critical flows first (search, cart, checkout, payment, login).
- **Compatibility matrix:** Execute the same designed test cases on every combination and check layout, functionality, forms, performance and security.
- **Execution:** Automated functional tests with Selenium Grid/Playwright and manual visual checks on real devices (BrowserStack, LambdaTest).
- **Defect reporting:** Record browser, version and OS in every defect.
- **Regression:** Retest all supported browsers after each fix, and repeat for each release.

How such issues are handled effectively

- Use web standards and validate the HTML and CSS.
- Use polyfills, vendor prefixes and responsive design; avoid unsupported features.
- Use visual regression tools to compare screenshots.
- Include browser compatibility in the definition of done and in the exit criteria.

Application to the Scenario

The "Place Order" button works in Chrome but not in Safari because of a JavaScript feature that Safari does not support. The compatibility matrix exposes the failure, a polyfill fixes it, and the retest on all browsers confirms it.

Conclusion

A planned, matrix-based and risk-prioritised strategy gives a consistent experience on all supported browsers and prevents customer dissatisfaction.

Question 19

A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

Answer

Concept

Validation (in the sense of input validation) is the checking of data against rules for format, range, length, type and mandatory fields before it is processed. Weak validation lets wrong data enter the system, which then produces inconsistent outputs. In testing terms, validation also means checking that the product meets the user's needs.

Gaps in validation

- The requirements did not define the valid formats and ranges (pin code, weight, date, phone number).
- Only valid, "happy path" inputs were tested; invalid, empty, boundary and special-character inputs were not.
- Cross-field rules were not checked, for example delivery date earlier than dispatch date.
- Validation was done in the interface but not on the server or in the database.
- There was no early review of the validation rules and no negative test cases.

How proper testing improves reliability

- Equivalence partitioning and boundary value analysis provide valid and invalid cases for every field.
- Negative testing confirms that wrong input is rejected with a clear message.
- Server-side, database and integration testing make sure that invalid data cannot reach the processing logic.
- Early review of validation rules finds the gaps before coding.
- Regression tests keep the validation working after changes.

Application to the Scenario

For a weight field accepting 0.1 to 1,000 kg, the tests are 0, 0.1, 0.2, 999.9, 1,000, 1,001, "abc", a negative value and a blank field. If the system accepts "-5" and calculates a negative shipping charge, the gap is found and corrected.

Conclusion

Systematic validation testing ensures that only correct data is processed, so the outputs are consistent and the system is reliable.

Question 20

A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

Answer

Concept

Treating testing as an engineering activity means following a defined process: test planning, test analysis, test design, implementation, execution, evaluation and closure. Test design is the step in which the requirements are converted into test conditions and test cases with inputs, preconditions, steps and expected results.

Improved process

- Analyse requirements and identify test conditions and risks.
- **Select suitable techniques:** equivalence partitioning, boundary value analysis, decision tables, state transition and use case testing (black box), and statement and decision coverage (white box).
- Document the test cases with IDs and expected results, and review them.
- Link the cases to requirements in a traceability matrix.
- Execute, log results and defects, and measure coverage.

Importance of systematic test design

- **Coverage:** All requirements are tested and gaps are visible.
- **Effectiveness:** Techniques select the most defect-prone values, so fewer tests find more defects.
- **Repeatability and reuse:** Cases can be rerun by anyone and used for regression.
- **Early defect detection:** Designing tests exposes errors in the requirements.
- **Measurement:** Coverage and defect metrics support decisions.
- **Efficiency:** Avoids duplicated or useless tests.

Application to the Scenario

For a field that accepts 1 to 100, random inputs such as 10 and 50 pass. The designed set 0, 1, 2, 99, 100, 101 finds a boundary defect with six cases.

Conclusion

Systematic test design replaces guesswork with a planned approach and gives higher defect detection with lower effort.

Question 21

A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

Answer

Concept

Data inconsistency means that the same data has different values in different modules, for example an allergy recorded in the Registration module that is missing in the Pharmacy module. In healthcare, this can

threaten patient safety, so the testing is risk-based and especially rigorous (testing is context dependent). Defect clustering suggests starting at the interfaces between modules.

Root causes

- **Requirement defects:** Data definitions, ownership and update rules were not clearly specified.
- **Design defects:** Duplicate data stores and no single source of truth.
- **Coding and interface defects:** Different data types or formats, wrong field mapping, and dependent records not updated.
- **Transaction and timing defects:** No rollback on partial failure, and concurrent updates overwriting each other.
- **Change defects:** A change in one module without impact analysis for the others.

How proper testing ensures consistency

- Integration testing follows a data item through the modules (Registration, Pharmacy, Billing, Laboratory).
- Database testing checks constraints, referential integrity and CRUD operations.
- Transaction tests check atomicity, rollback and concurrent updates.
- Reconciliation tests compare the data of the different modules.
- End-to-end scenario tests and regression tests after each change.

Application to the Scenario

Scenario: register a patient, record a penicillin allergy, prescribe a medicine, and check the allergy alert in Pharmacy and the summary in Billing. If Pharmacy does not show the allergy, an interface defect is reported with clear evidence.

Conclusion

Testing the data flow between modules finds inconsistencies before release, which protects data integrity and patient safety.

Question 22

A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

Answer

Concept

Quality assurance cannot be added at the end of a project. Testers should take part in every phase of the SDLC, so that defects are prevented or found as early as possible and test preparation proceeds in parallel with development.

Role of testers across the SDLC phases

SDLC phase	Role of the tester
Requirements	Review for clarity, completeness and testability; identify risks; begin the test plan and acceptance criteria
Design	Review the design and interfaces; design test cases and test data; plan the test environment and tools

SDLC phase	Role of the tester
Coding	Support unit and integration testing; perform static analysis and code reviews; prepare automation scripts
Testing	Execute test cases, report defects, retest fixes and run regression tests; report status and metrics
Deployment / Maintenance	Smoke and acceptance testing; regression testing for changes; analyse production defects

How early involvement improves quality

- Defects in requirements and design are found when they are cheapest to fix.
- Test cases are ready when coding ends, so the test phase is shorter.
- Developers and testers share the same understanding of the requirements.
- Risks are known early, and so the schedule and estimates are more accurate.
- Collaboration practices (joint reviews, daily stand-ups, three-amigos sessions) reduce misunderstandings.

Application to the Scenario

In a library system, a tester asks in the requirement review what should happen when a member exceeds the borrowing limit or has an unpaid fine. The rules are added before coding. Otherwise the developer would have guessed, and the defect would have been found late, causing rework and delay.

Conclusion

Involving testers in all phases, especially the early ones, shortens the schedule, reduces rework and raises the quality of the product.

Question 23

A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

Answer

Concept

Test coverage is the extent to which requirements, scenarios, input conditions or code have been exercised by the tests. The principles "exhaustive testing is impossible" and "testing is context dependent" mean that tests must be chosen according to risk and to the way the system is actually used, including its peak load.

Gaps identified

- Only functional testing with one or a few users; no load, stress, spike or endurance testing.
- Peak scenarios (month-end, sales, exams) and performance targets were not defined in the test plan.
- The test environment and data were much smaller than in production.
- No risk analysis or monitoring of response time, CPU, memory and database connections.
- Failure and recovery scenarios (server restart, time-out) were not tested.

How better coverage prevents failures

- Scenario coverage includes non-functional cases derived from real usage data.
- Load testing at the expected peak confirms that targets are met.
- Stress testing beyond the peak finds the breaking point and checks graceful recovery.
- Endurance testing finds memory leaks, and spike testing handles sudden increases.

- Results are used to tune the system (caching, indexing, load balancing), and the tests are repeated for each release.

Application to the Scenario

In a simulated online examination with 3,000 students submitting answers in the same minute, JMeter shows that the response time rises to 20 seconds and the connection pool is exhausted at 1,500 users. After tuning, the retest meets the 3-second target, and the failure does not occur in the real examination.

Conclusion

Better coverage that includes realistic peak scenarios exposes capacity problems before release, and prevents failures during peak usage.

Question 24

A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

Answer

Concept

A defect repository is a central database in which every defect is recorded in a standard format and tracked through its life cycle. Without it, defects are held in e-mails, chats and memory; they are lost, duplicated or forgotten, and no analysis is possible.

Applying the repository concept

- Adopt a defect tracking tool (Jira, Bugzilla, Mantis) as the single official place for defects.
- **Use a standard report template:** ID, title, module, version, environment, steps to reproduce, expected and actual results, severity, priority, status, attachments, owner, root cause and fix details.
- **Define a defect life cycle:** new, assigned, fixed, retested, closed or reopened (and rejected or deferred).
- Use classification (type, origin, severity) for every defect, following IEEE 1044.
- Review the reports for completeness and duplicates in regular triage meetings.

How documentation enhances tracking and analysis

- **Tracking:** Every defect has a known owner and status, so none is lost.
- **Traceability:** Links to the requirement, test case and code change show the impact.
- **Faster fixing:** Developers can reproduce the defect from complete steps and logs.
- **Analysis:** Structured fields allow charts of defects by module, type, severity and phase, defect density, ageing and reopen rate.
- **Decisions:** Trends show weak modules and processes and help to decide release readiness.

Application to the Scenario

The report "Login not working" is replaced by "LG-102: login fails with a valid user ID containing a capital letter. Steps: ..., Expected: login success. Actual: error 401. Severity: major". The developer reproduces it in minutes, and the repository shows that three similar defects exist in the Login module.

Conclusion

A structured, well-documented defect repository makes defect management organised and measurable, and provides data for continuous improvement.

Question 25

A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

Answer

Concept

The seven testing principles (testing shows the presence of defects, exhaustive testing is impossible, early testing, defect clustering, pesticide paradox, testing is context dependent, absence-of-errors fallacy) guide the design of an effective strategy. Transaction processing is a high-risk area, so the testing must be rigorous and organised.

Redesigned approach

- **Early testing:** Review transaction rules, limits and failure handling before coding.
- **Risk-based testing and defect clustering:** Give priority to transfer, payment and standing instruction modules.
- **Exhaustive testing is impossible:** Use equivalence partitioning, boundary value analysis and decision tables.
- **Pesticide paradox:** Review and add test cases every release, including cases for each past failure.
- **Context dependent:** Add security, data integrity, concurrency, performance and audit tests.
- Standard process, defect repository, test management tools and automated regression in continuous integration.

How structured testing enhances reliability

- Each business rule has designed test cases, so coverage is complete and measurable.
- Results are repeatable, and defects are tracked until closed.
- Failures such as duplicate debit, partial transactions and wrong balance are detected before release.
- Defect analysis removes root causes, so failures do not recur.

Application to the Scenario

Test case	Expected result
Transfer equal to the available balance	Accepted, balance becomes zero
Transfer of balance plus Rs.1	Rejected with a clear message
Failure after debit and before credit	Complete rollback; no money lost
Two simultaneous transfers on one account	Balance consistent; no overdraft
Submit pressed twice	Only one transaction processed

Conclusion

Structured testing based on proven principles gives dependable transaction processing and reduces repeated failures.

Question 26

A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.

Answer

Concept

Edge cases are inputs or situations at the extremes of the operating range or outside normal use: minimum and maximum values, empty or null input, very long text, special characters, zero, negative numbers, and special dates. Testers who use only normal inputs follow the "happy path", and so miss them. The principles "exhaustive testing is impossible" and "pesticide paradox" show that tests must be chosen deliberately and updated.

Including edge cases in testing

- Define the limits explicitly in the requirements, and identify the equivalence classes.
- Add boundary, negative and unusual tests for every input and for every rule.
- Include empty and null values, maximum field length, special characters, zero and negative numbers, leap day (29 February), month-end and time-zone changes.
- Use error guessing and the defect repository (past defects) to create edge-case checklists.
- Review test cases regularly, and use data-driven automation for edge values.

How boundary testing improves defect detection

Most defects occur at the edges of ranges because of wrong operators (< instead of <=), off-by-one errors in loops and wrong limits. Boundary value analysis tests the values just below, on and just above each boundary. With only a few cases it covers the places where defects are most likely, and gives high detection at low cost.

Application to the Scenario

For a withdrawal of Rs.100 to Rs.20,000, the boundary tests are 99, 100, 101, 19,999, 20,000 and 20,001. A defect that wrongly rejects Rs.20,000 would be found by these tests, but not by normal values such as Rs.5,000.

Conclusion

Including edge cases using boundary testing finds the defects that normal inputs miss and prevents failures after deployment.

Question 27

A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

Answer

Concept

A process-based (process-oriented) testing approach defines one common, documented test process for the whole organisation, and improves it continuously. Models such as ISO/IEC 29119, TMMi and the ISTQB fundamental test process give the framework. When teams test in their own different ways, quality is uneven and results cannot be compared.

Applying process-based testing

- **Define a common test process:** planning, analysis and design, implementation, execution, reporting and closure.
- Create standard templates for the test plan, test case, test report and defect report, and a common set of tools.
- Set the same entry and exit criteria and coverage targets for all modules.
- Hold reviews and quality audits to confirm that the process is followed, using checklists.
- **Collect uniform metrics:** requirement coverage, defect density and defect removal efficiency.
- Run retrospectives and update the process (continuous improvement).

How standardization improves outcomes

- Every module receives the same level of testing, so quality is even.
- Results and metrics are comparable, and weak areas become visible.
- Reusable templates and test assets save time and reduce errors.
- New members become productive quickly, and quality does not depend on individuals.
- Management can plan and predict more accurately.

Application to the Scenario

The Payroll team used boundary and decision-table design and a tracking tool, while the Leave team tested informally and found fewer defects. After a common test plan template and the exit criterion "no open critical defects and 90% requirement coverage" were adopted, defects found in system testing of the Leave module increased and production defects fell.

Conclusion

Standardised, process-based testing removes inconsistency between modules and gives uniform, predictable quality.

Question 28

A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

Answer

Concept

Input validation checks that the data given by the user conforms to the allowed format, type, range and length before it is processed. Robustness is the ability of a system to continue to work correctly, or to fail gracefully, when it receives invalid or unexpected input or conditions.

Improving input validation through testing

- Specify valid and invalid values for every field in the requirements (type, range, length, mandatory, format).
- Apply equivalence partitioning and boundary value analysis to create valid and invalid classes.
- **Add negative tests:** blank, null, wrong data type, very long text, special characters, spaces, SQL injection and script (XSS) strings, and wrong file types or sizes.
- Test validation on the client side and also on the server side, because client checks can be bypassed.

- Check that error messages are clear and that no internal information is displayed.
- Use exploratory testing and fuzzing to find unforeseen cases.

How proper testing ensures robustness

Negative and security tests prove that bad data is rejected safely, without crash, data corruption or security breach. Boundary tests confirm the limits, and regression tests make sure that the validation continues to work after changes. The application therefore behaves predictably in all situations, including misuse.

Application to the Scenario

Entering "abc" or "-5" in an age field must give the message "Enter a valid age between 18 and 60" and must not cause a server error or store the value. Entering a script string in a name field must be blocked.

Conclusion

Systematic input validation testing makes the application robust, secure and reliable.

Question 29

A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

Answer

Concept

Defect leakage is the percentage of defects that escape testing and are found in production (or in a later phase): leakage = (defects found after release / total defects) x 100. High leakage shows that the testing process is not effective.

Root-cause analysis using testing principles

- Testing shows the presence, not the absence, of defects: no exit criteria based on risk, so testing stopped too early.
- **Incomplete coverage:** unclear or untested requirements, and missing boundary, negative and non-functional tests.
- **Pesticide paradox:** the same tests were repeated and the regression suite was out of date.
- Test environment and data different from production.
- Late or no involvement of testers, and schedule pressure that reduces testing time.
- Production defects were not analysed, so the lessons were lost.

Improved testing practices

- Early testing, requirement traceability and risk-based prioritisation.
- Techniques such as BVA, decision tables and exploratory testing; regular updating of test cases.
- Production-like environments and data; automated regression and smoke tests.
- Defined exit criteria and defect metrics (leakage, defect removal efficiency).
- Root-cause analysis of every leaked defect, followed by a new test case and a process change.

Application to the Scenario

If analysis shows that most leaked defects are in the Payment module under special data conditions, the team adds data-driven boundary and negative tests, tests with production-like data, and tracks the leakage rate until it falls.

Conclusion

Analysing root causes and applying disciplined testing practices detect more defects before release, and so reduce defect leakage into production.

Question 30

A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.

Answer

Concept

A test strategy defines the approach to testing, in terms of levels, techniques, timing, tools and responsibilities. The principle of early testing says that testing activities must start in the early phases, and the principles of defect clustering and risk-based testing say that effort must go where defects are most likely.

Redesigned strategy

- **Shift-left:** Testers join requirement and design reviews and sprint planning; static testing (reviews, walkthroughs, inspections) is used from the start.
- Developers perform unit testing, code reviews and static analysis.
- Continuous integration with automated unit and smoke tests on every build.
- **Risk-based and defect-clustering approach:** test high-risk and defect-prone modules first.
- Define entry and exit criteria for every phase, and analyse defect-repository data to improve test design.
- Track metrics such as the phase in which defects are found (defect removal efficiency).

How early detection improves efficiency

- The cost of correcting a defect rises steeply in later phases, because design, code and tests must all be changed.
- Rework, delay and effort are reduced, and the schedule is more predictable.
- Fewer defects reach the later test phases, so they are shorter and more stable.
- Developers fix defects while the code is still fresh in their mind.
- Total cost falls, and the quality of the final product increases.

Application to the Scenario

An ambiguous requirement found in a review takes minutes to correct. The same defect found in production needs analysis, redesign, recoding, retesting, redeployment and handling of customer complaints, which may take days and cost many times more.

Conclusion

A strategy based on early testing and risk focus finds defects when they are cheapest to fix, which reduces cost and effort and improves efficiency.